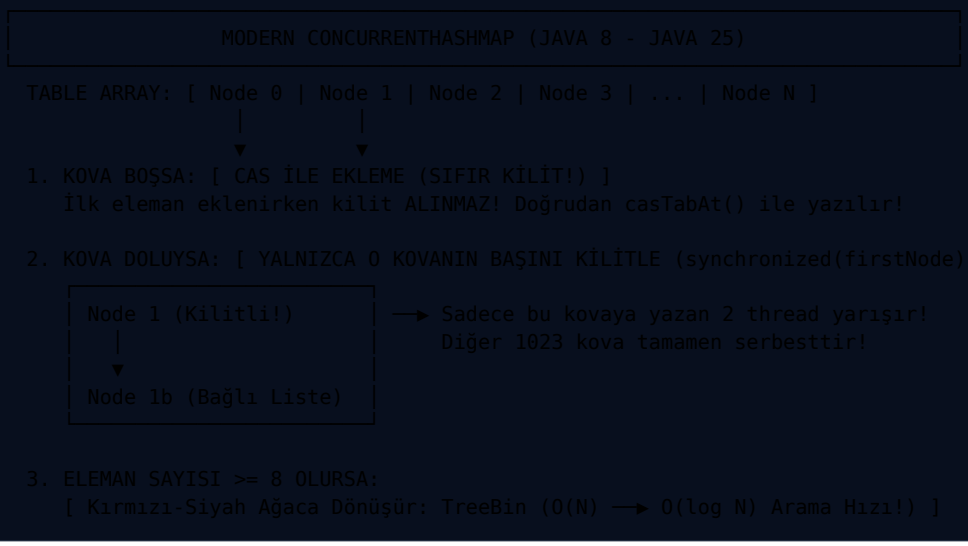
31

BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

ConcurrentHashMap İç Mimarisi: Segment Yok! Node CAS, Bin Kilitleri & TreeBin

ConcurrentHashMap Internals: Node CAS, Bin-Level Locking & TreeBins

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski usul senkronize haritalar (Hashtable), 1000 çekmeceli bir kütüphanenin kapısına tek bir kilit takmak gibidir. Biri 1. çekmeceden kitap alırken diğer 999 kişi kapıda bekler. ConcurrentHashMap ise her çekmecenin başına küçük bir şifreli kilit takar! Biri 1. çekmeceyi açarken, diğeri 500. çekmeceden rahatça kitabını alır; birbirlerini hiç görmezler. Eğer bir çekmecede çok fazla dosya birikirse, çekmece otomatik olarak açılıp alfabetik bir fihrist ağacına (TreeBin) dönüşür!

🇬🇧 ENGLISH (ELI5)

A synchronized map (`Hashtable`) locks the entire bank vault whenever someone accesses safety deposit box 1 ConcurrentHashMap locks ONLY safety deposit box 1 (`synchronized(firstNode)`). Customers accessing boxes 2 through 1024 proceed concurrently with ZERO friction! Even better: if a box is empty, it inserts the item using atomic lock-free CAS without locking at all! If a single bucket accumulates more than 8 colliding items, it dynamically transmutes into a Red-Black Tree (`TreeBin`), preserving O(log N) search speed.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 8+ ve Java 25 ConcurrentHashMap İnovasyonları: 1) Segmentlerin Kaldırılması: Java 7'deki 16'lı `Segment[]` mimarisi çöpe atıldı. Artık doğrudan `Node\<K,V>[] table` dizisi kullanılır. 2) CAS ile İlk Ekleme: Kova (Bucket) boşsa, `Unsafe.compareAndSetObject` (modern JDK'da VarHandle) kullanılarak kilit almadan tek çevrimde yazılır. 3) Kova Düzeyinde Kilitleme: Kova doluysa sadece o kovanın ilk elemanı (`synchronized(node)`) kilitlenir; haritanın geneli açık kalır. 4) Treeify Eşiği: Tek bir kovadaki eleman sayısı 8'i geçerse ve tablo boyutu >= 64 ise bağlı liste kırmızı-siyah ağaca (`TreeBin`) dönüştürülür. Eleman sayısı 6'ya düşerse tekrar listeye (Untreeify) döner. 5) size() Hesabı: LongAdder mantığında `CounterCell[]` kullanılarak kilit almadan hesaplanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot ConcurrentHashMap Topology: 1) Demise of Segmented Locking: Java 8 discarded the historical 16-Segment structure. The map operates directly on a volatile `Node\<K,V>[] table`. 2) CAS-Driven Empty Bucket Ingestion: If `tabAt(i) == null`, the entry is inserted via `casTabAt()` with zero lock acquisition. 3) Bin-Level Synchronized Locks: When collisions occur, the thread synchronizes strictly on the root `Node` of that specific hash bin. Concurrent threads hashing to distinct bins execute in parallel. 4) TreeBin Transformation: If bin collisions reach `TREEIFY_THRESHOLD = 8` and table capacity >= 64, the singly-linked list transmutes into a balanced Red-Black Tree (`TreeNode`), bounding worst-case lookup from $O(N)$ to O(log N). 5) High-Concurrency Size Aggregation: `mappingCount()` leverages internal `CounterCell` arrays, mirroring `LongAdder` to prevent CAS contention.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ConcurrentHashMap ile Atomik computeIfAbsent ve İstatistikler
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.LongAdder;

public class HighThroughputUrlTracker {
    // URL bazlı istek sayaçları
    private final ConcurrentHashMap<String, LongAdder> accessCounters = new ConcurrentHashMap<>();

    public void trackVisit(String url) {
        // ⚠ ATOMİK: URL yoksa LongAdder oluştur, varsa eskisini getir (KİLİTSİZ!)
        accessCounters.computeIfAbsent(url, _ -> new LongAdder()).increment();
    }

    public long getVisitCount(String url) {
        var adder = accessCounters.get(url);
        return adder != null ? adder.sum() : 0L;
    }

    void main() {
        var tracker = new HighThroughputUrlTracker();
        tracker.trackVisit("https://maang.com/careers");
        tracker.trackVisit("https://maang.com/careers");
        IO.println("Sayfa Ziyaret Sayısı: " + tracker.getVisitCount("https://maang.com/careers"));
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı CHM Tuzağı: 'ConcurrentHashMap içine `null` key veya `null` value koyabilir miyiz?' Yanıt: 'KESİNLİKLE HAYIR! HashMap null kabul eder ama ConcurrentHashMap null key veya null value'ya ASLA izin vermez (NPE fırlatır). Çünkü çoklu thread ortamında map.get(k) == null döndüğünde 'Anahtar gerçekten yok mu, yoksa değeri mi null?' belirsizliği (ambiguity) kilit-siz çözülemez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory Interview Axiom: 'Can ConcurrentHashMap store null keys or null values?' Senior Answer: 'NEVER! `HashMap` permits nulls, but `ConcurrentHashMap` explicitly forbids null keys and null values, throwing `NullPointerException`. Doug Lea designed this because in a multithreaded map, `map.get(key) == null` cannot disambiguate whether the key is missing or the value is legitimately null without holding a lock across `containsKey()`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentHashMap boş kovalara kilit almadan CAS ile eleman ekler.
- Kova dolduğunda tüm tablo değil, sadece o kovanın ilk elemanı kilitlenir.
- ConcurrentHashMap null key ve null value kabul etmez (NullPointerException).

🎯 Key Takeaways for the Interview

- ConcurrentHashMap inserts into empty bins via lock-free CAS operations.
- Only the root node of the target hash bin is synchronized during collisions.
- ConcurrentHashMap strictly forbids null keys and values to prevent race ambiguities.